

## Autonomous I (Group B )

Mohamed Abdo,<sup>1</sup> Arya Karimi Jafari,<sup>2</sup> Mustafa Qaiser<sup>3</sup>

### Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Motivation</b>                           | <b>2</b>  |
| <b>2</b> | <b>Individual Scenarios</b>                 | <b>2</b>  |
| 2.1      | Safety scenario . . . . .                   | 2         |
| 2.2      | Route Change Scenario . . . . .             | 5         |
| 2.3      | Merging Truck Scenario . . . . .            | 7         |
| <b>3</b> | <b>Complete UPPAAL Model</b>                | <b>9</b>  |
| <b>4</b> | <b>SVM Implementation</b>                   | <b>12</b> |
| 4.1      | Carla Dataset Filtering . . . . .           | 12        |
| 4.2      | SVM Classification and Evaluation . . . . . | 13        |
| <b>5</b> | <b>Conclusion</b>                           | <b>14</b> |
| <b>6</b> | <b>Declaration of Originality</b>           | <b>14</b> |

---

<sup>1</sup> mohamed.abdo@stud.hshl.de

<sup>2</sup> arya.karimi-jafari@stud.hshl.de

<sup>3</sup> muhammad-mustafa.qaiser@stud.hshl.de

### **Abstract:**

This project explores autonomous truck platooning by modeling key driving behaviors such as emergency safety, route change, and merging using formal verification and visual lane detection techniques. Critical scenarios are described and implemented in UPPAAL using Timed Automata to ensure safety, liveness, and deadlock freedom. Each vehicle's logic is modeled with broadcast channels and synchronization signals, enabling timed coordination. In parallel, a Support Vector Machine (SVM) is trained on lane markings from CARLA simulation data to classify lane positions and convert visual input into real-time steering decisions. By combining formal verification with machine learning-based control, the system demonstrates how autonomous trucks can operate reliably and safely under varying conditions.

## **1 Motivation**

Truck platooning improves road efficiency by allowing heavy vehicles to travel closely in coordinated groups, reducing drag and saving fuel. One of the most complex operations in this system is highway merging, where a truck or new platoon must safely join an existing one. This requires real-time coordination, safe gap creation, and strict timing control. Studies like [As23] and [IC23] highlight the importance of maintaining time gaps between 0.8 and 1.2 seconds to balance safety and performance.

In parallel, lane detection is essential for vehicle awareness and path planning. Classical computer vision techniques such as contour analysis [Op23b] and centroid estimation [Ma21] can extract key features from road images. These features are then used by machine learning models like Support Vector Machines (SVMs) [Op23a] to classify lane position. This project integrates SVM-based vision with UPPAAL formal modeling to address both visual perception and verified behavior for safe, autonomous merging and lane following.

## **2 Individual Scenarios**

Each team member contributed a separate UPPAAL scenario, which was verified individually. These scenarios were later merged into a single, complete model.

### **2.1 Safety scenario**

Safety is the most important feature that needs to be taken into consideration because the system involves close following distances between heavy vehicles traveling at high speeds. Any malfunction or miscommunication could lead to serious accidents involving not only the platoon, but also surrounding vehicles in the highway. Therefore, robust safety measures must be built into every layer of the system.

The behavior of the leader truck is determined by the distance to the obstacle that is chosen by the *Environment* template. If the obstacle is greater than 35 meters, then the leader, and

all follower trucks remain idle. However, if the obstacle is set at a distance between 15 and 35 meters, then the leader truck performs a lane change to safely avoid the obstacle, all follower trucks also perform the lane change as they are following the leader at all times. If the obstacle is less than 15 meters away, then all trucks in the platoon will perform an emergency stop as the leader is obliged to come to an immediate stop.

In addition to the obstacle related behavior, the model includes a safety condition based on the gap between the trucks that are found within the platoon. If at any case the gap between the trucks falls below 10 meters, then all follower trucks are forced into an emergency stop to avoid collisions and accidents.

The behavior and structure of the safety scenario have been modeled using several key SysML diagrams. The activity diagram highlights the overall function of the safety scenario, as found in Figure 4. The block diagram found in Figure 6 shows the main components, while Figure 9 and Figure 5 depict the requirements and sequence diagrams of the model.

The safety scenario is modeled in UPPAAL using timed automata and synchronizing broadcast channels which allow the follower trucks to constantly follow the leader truck's actions as found in Figure 1. The actions of the UPPAAL model have been described in the previous text, and time features have been included to ensure the trucks are performing their tasks adequately and correctly.

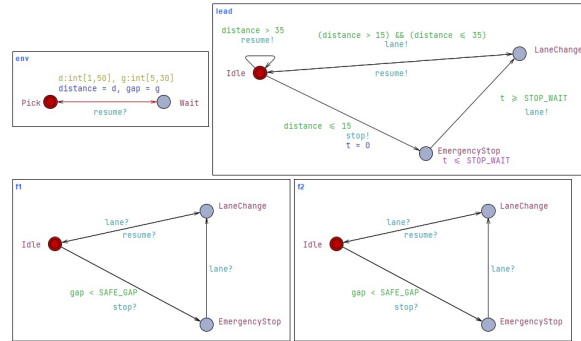


Fig. 1: Safety Scenario UPPAAL Model

```
const int SAFE_GAP = 10;
const int STOP_WAIT = 5;
int distance = 0;
int gap = 20;
broadcast chan lane;
broadcast chan stop;
broadcast chan resume;
```

Fig. 2: Declarations

| Status                                                           |
|------------------------------------------------------------------|
| A[] not deadlock                                                 |
| Verification/kernel/elapsed time used: 2.438s / 0.016s / 2.479s. |
| Resident/virtual memory usage peaks: 22,456KB / 72,364KB.        |
| Property is satisfied.                                           |

Fig. 3: Deadlock Verification

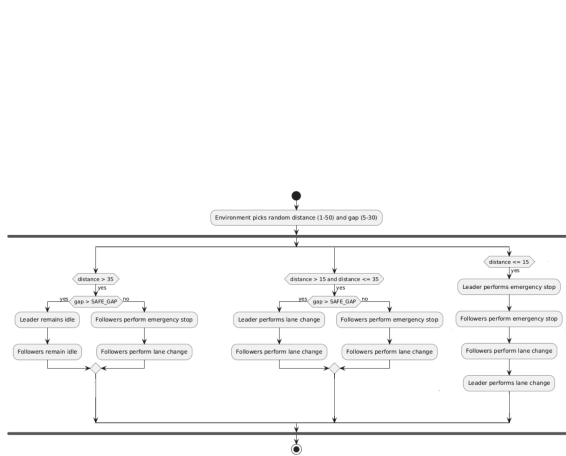


Fig. 4: Activity Diagram

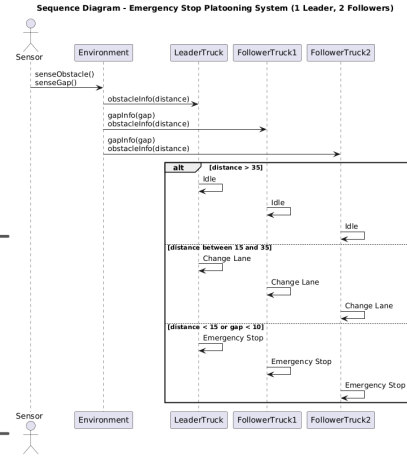


Fig. 5: Sequence Diagram

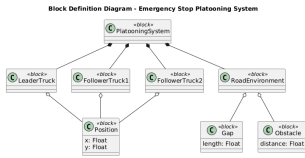


Fig. 6: Block Definition Diagram

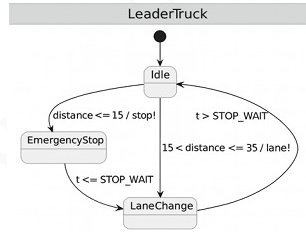


Fig. 7: Leader Truck Stm

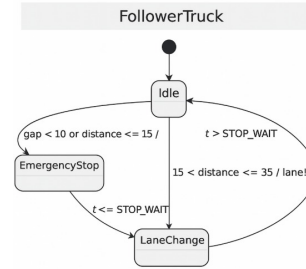


Fig. 8: Follower Truck Stm

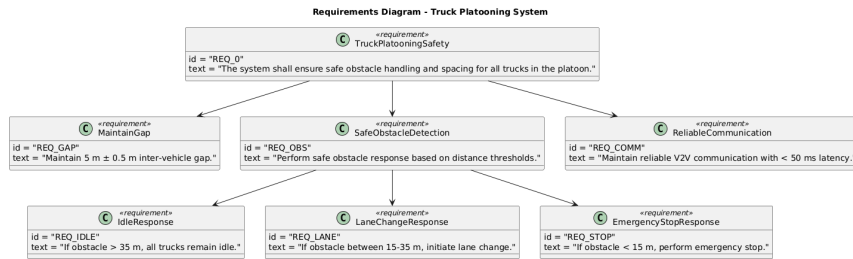


Fig. 9: Requirements Diagram

## 2.2 Route Change Scenario

In modern freight logistics, route changes allow autonomous platoons to adapt to real-time traffic updates and delivery deadlines. Unlike safety-focused merges, this scenario optimizes operational efficiency. For example, if a rear truck carries a time-sensitive load, it may need to shift forward in the formation to meet delivery targets [Fr95].

The process begins when the lead truck broadcasts a rerouting signal and proposes a new formation. Each vehicle evaluates spacing, speed, and feasibility. If accepted, the rear truck overtakes to reach the middle position while the others adjust to create a safe gap. Timing coordination is enforced using synchronized clocks.

This behavior is modeled in UPPAAL using timed automata and broadcast channels. The complete model is shown in Figure 10, and its correctness is verified for timing and deadlock freedom using declarations and properties Figure 11.

System-level behavior is represented in SysML diagrams. The activity flow is shown in Figure 12, while inter-vehicle message exchange is captured in the sequence diagram Figure 13. The structural layout of truck roles and components appears in the block definition diagram Figure 14. Figures 15 and 16 detail the control logic of the Middle and Merging Trucks. Finally, Figure 17 summarizes the system requirements and constraints.

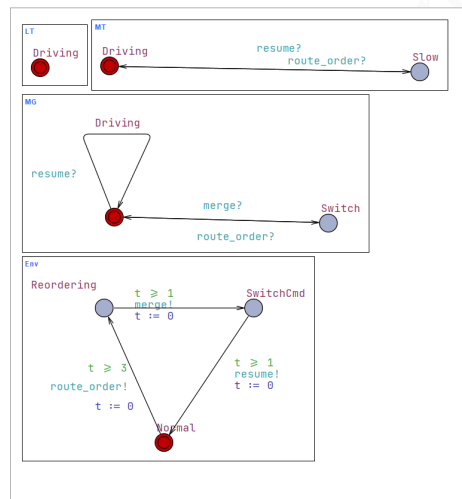


Fig. 10: UPPAAL model for route reconfiguration

**Declarations:**  
 broadcast chan route\_order;  
 broadcast chan merge;  
 broadcast chan resume;

**System Declarations:**  
 env = Env();  
 lt = LT();  
 mt = MT();  
 mg = MG();  
 system env, lt, mt, mg;

Status  
 A[] not deadlock  
 Verification/kernel/elapsed time used: 0s / 0s / 0.004s.  
 Resident/virtual memory usage peaks: 17,764KB / 62,220KB.  
 Property is satisfied.

Fig. 11: Declaration and deadlock verification

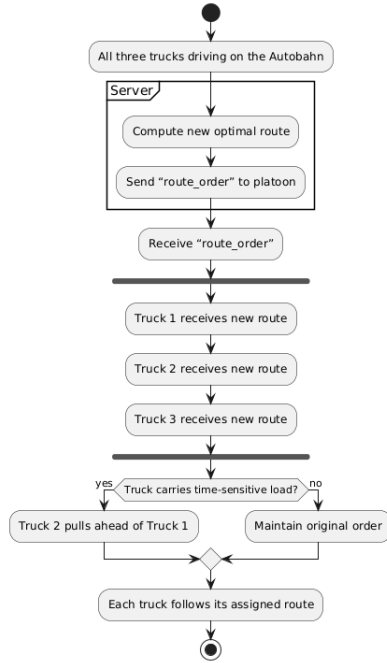


Fig. 12: Activity Diagram

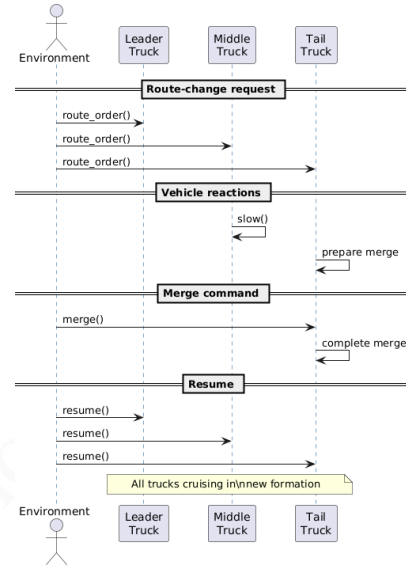


Fig. 13: Sequence Diagram

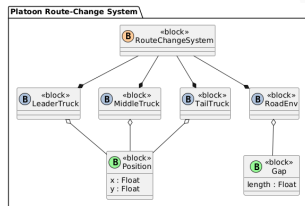


Fig. 14: Block Definition Diagram

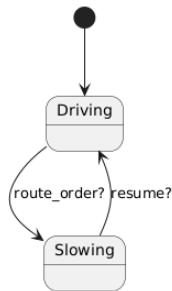


Fig. 15: Middle Truck Stm

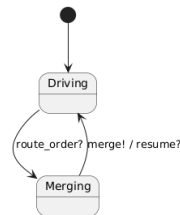
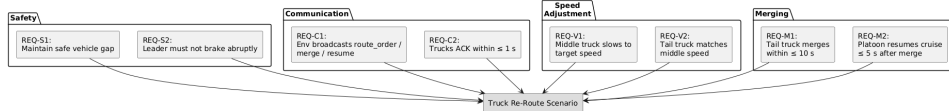


Fig. 16: Merging Truck Stm



1: Requirements Diagram

### 2.3 Merging Truck Scenario

On highways, a merging truck must safely join a platoon from an on-ramp. This requires coordination and a temporary increase in the gap between vehicles [As23]. A gap of 0.8 to 1.2 seconds is typically recommended to ensure both safety and fuel efficiency [IC23].

The merging process starts when the truck sends a `merge_request`. If accepted, the follower truck slows down and the leader replies with `merge_ack`. Once the gap is ready, the truck sends `start_merge` and enters the platoon. Timing checks like  $t \geq \text{min\_gap\_time}$  and flags like `gap_ready` ensure safe merging. If no response is received, the merge is aborted or retried.

The behavior must be tightly synchronized to avoid unsafe overlaps and ensure fairness in execution timing. All transitions are guarded by both clock values and shared channel events to prevent race conditions.

This behavior is modeled in UPPAAL using timed automata and broadcast channels. The full model is shown in Figure 18, the signal exchange in Figure 19, and the deadlock verification in Figure 20.

SysML diagrams support the design: the activity diagram Figure 21 shows the merging flow, while Figure 22 shows system structure. Requirements and communication are shown in Figures 23 and 24. Each vehicle's behavior is modeled using state machines: the follower Figure 25, the merging truck Figure 30, and the leader Figure 27.

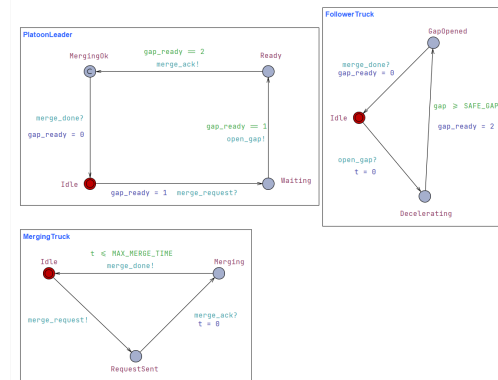


Fig. 18: UPPAAL timed automata model

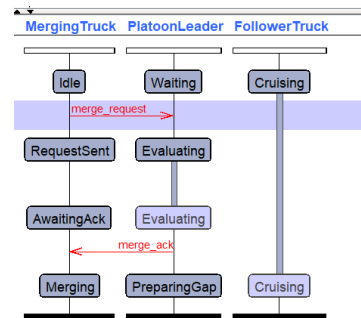


Fig. 19: Merge Request

| Status                                                    |
|-----------------------------------------------------------|
| A[] not deadlock                                          |
| Verification/kernel/elapsed time used: 0s / 0s / 0.008s.  |
| Resident/virtual memory usage peaks: 18,616KB / 63,904KB. |
| Property is satisfied.                                    |

Fig. 20: Deadlock Check





### 3 Complete UPPAAL Model

This section of the documentation will focus on the complete UPPAAL model which combines the three introduced scenarios. As safety has been identified as the most important feature of the platoon, it was decided to build the merged UPPAAL model revolving around the safety scenario model. New templates and states were added to the model in order to accommodate the three scenarios together. The *Environment* template received a new state called *NewRoute* which can only be triggered when the distance to the obstacle is greater than 35 meters, and the safety gap has been acknowledged. By arriving to this state, scenario 2 gets activated where *Follower1* must enter the *SlowDown* state, and allow *Follower2* to perform its two proceeding states - *LaneChangeLeft* & *LaneChangeRight* - in order to overtake the first follower, and stay in the lead as it has time sensitive cargo. In addition, when all safety requirements have been met, Scenario 2 is also introduced, where the *MergingTruck* is able to send a *JoinRequest* and then join the platoon by transitioning from the previous state to *PlatoonJoined*, throughout this model Timed Automata and synchronizations have been used in order to ensure that all procedures are working in chronological order, representing a real platoon. In cases where the distance to the obstacle is not greater than 35 meters, or the gap between the vehicles is less than the assigned *SAFE\_GAP*, the original safety scenario model takes over, with the Leader Truck, and Follower trucks performing a lane change when the obstacle is between 15 - 35 meters, and an emergency stop when the obstacle is closer than 15 meters. If at any point the *SAFE\_GAP* requirement is not met, then the Follower trucks and Merging Truck will perform an emergency stop, to avoid any collisions.

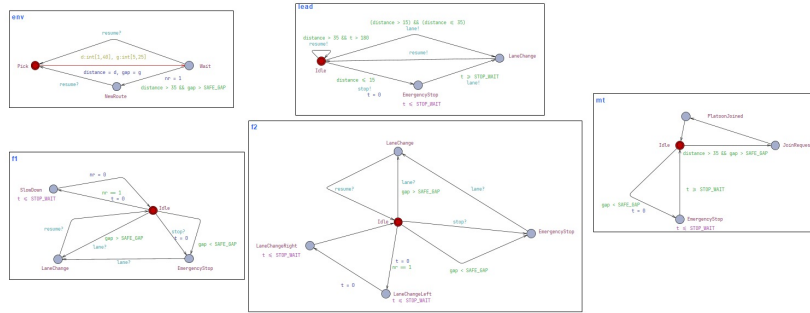


Fig. 28: Combined UPPAAL Model

```
const int SAFE_GAP = 10;
const int STOP_WAIT = 5;
int distance = 0;
int gap = 20;
int nr = 0;
broadcast chan lane;
broadcast chan stop;
broadcast chan resume;
```

Fig. 29: Declarations

```
env = Environment();
lead = Leader();
f1 = Follower1();
f2 = Follower2();
mt = MergingTruck();
env, lead, f1, f2, mt;
```

Fig. 30: System Declarations

```
Status
[All] not deadlock
Verification/kernel/elapsed time used: 167.719s / 0.578s / 170.026s.
Resident/virtual memory usage peaks: 115,024KB / 257,540KB.
Property is satisfied.
```

Fig. 31: Deadlock Verification

Presented below are the sequence diagrams depicting the different cases within our model. As previously mentioned when introducing the safety scenario, the *Environment* picks a value for the distance between the *Leader* truck and an obstacle ahead, as well as a random safety gap value found between the *Followers*. Based on these values, our model will take the right course of action, in order to ensure safety and efficiency.

Figure 32 highlights what occurs when an obstacle has been detected close by ( $\leq 15$  meters), without gap issues, while Figure 33 shows how if the obstacle is close by, and there are gap issues then the merging truck also performs an immediate emergency stop. These sequence diagrams help the reader visualize the synchronizations, and the timed automata in terms of the different trucks acting in chronological order.

Figure 34 focuses on showing the case where the obstacle has been detected closer than 35 meters, but further than 15 meters away, therefore all trucks perform a lane change with the help of the lane broadcast channel. On the other hand Figure 35, depicts that obliging to the safety gap has the highest priority as even though the obstacle is greater than 15 meters away, the follower and merging trucks still perform an emergency stop to avoid any collisions. The use of "guards" within the system allowed us to give absolute priority to safety at all times.

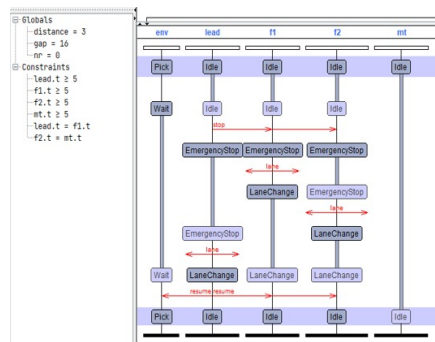


Fig. 32: Obstacle Close &amp; No Gap Issues

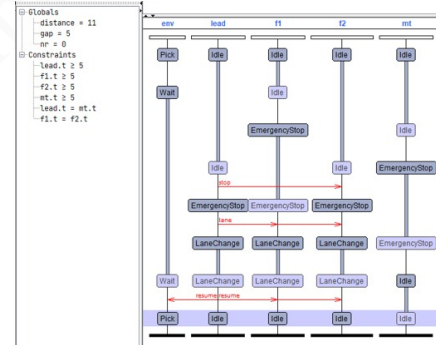


Fig. 33: Obstacle Close &amp; Gap Issues

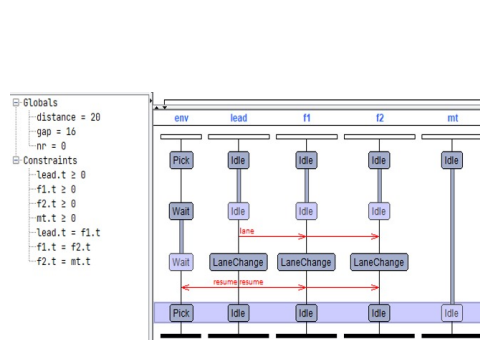


Fig. 34: Lane Change &amp; No Gap Issues

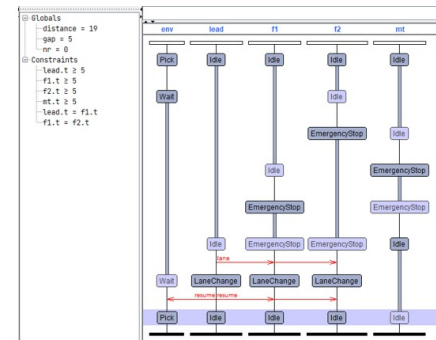
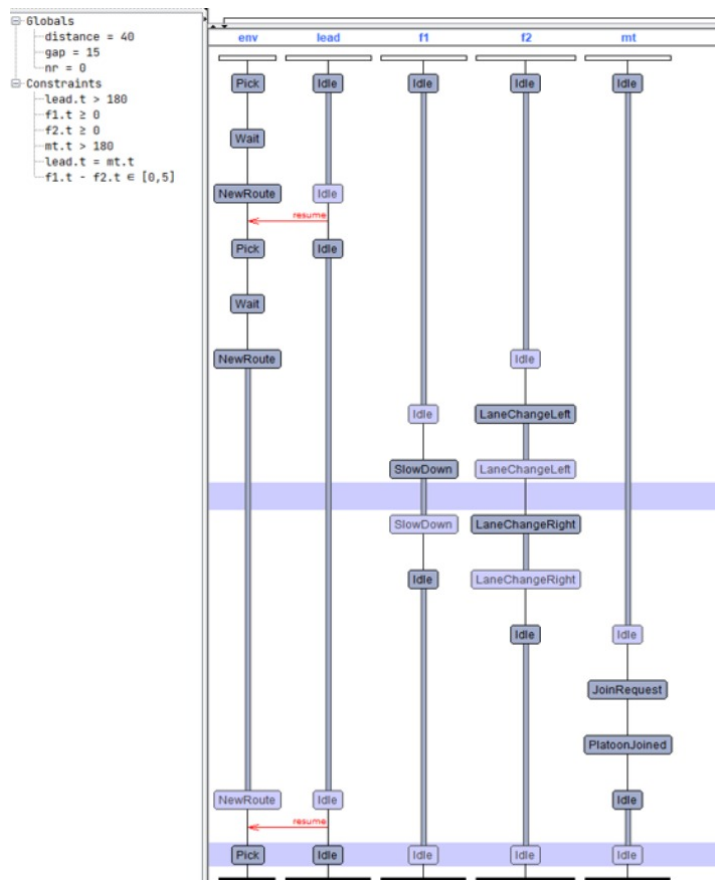


Fig. 35: Lane Change &amp; Gap Issues



The figure found above highlights the careful use of timed automata as it shows that the *Environment* has picked the *NewRoute* state, and this tells *Follower2* to perform a lane change to the left, and at the same time *Follower1* slows down for 5 seconds which is the set *STOP\_WAIT* configuration, and after these 5 seconds, *Follower2* returns to the main lane by performing a lane change to the right, successfully completing the overtaking maneuver. After the repositioning of the trucks, the *Merging Truck* can be seen sending a request and eventually joining the platoon.

## 4 SVM Implementation

To enable autonomous lane following, a Support Vector Machine (SVM) model was trained using visual input from the CARLA simulator. The goal was to classify the lane position left, center, or right from camera images and translate that into steering commands. The pipeline included image preprocessing, SVM training, and real-time simulation.

### 4.1 Carla Dataset Filtering

Raw images from the CARLA simulator were preprocessed to extract lane relevant features. The pipeline focused on detecting green and blue lane markings typically used in the environment. Only the bottom half of each image was processed to minimize noise and emphasize the road surface. As shown in Figure 37, the upper image is the original frame and the lower one is the cropped version used for detection. This region was converted from BGR to HSV color space for more accurate color filtering. A summary of the filtered dataset is shown in Table 1, The average x-position of non-zero mask values defined the estimated lane center, as shown in Equation 1: [Ma21]

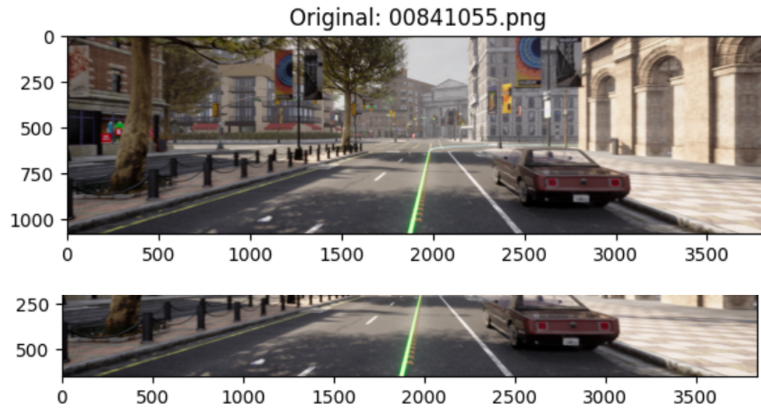


Fig. 37: Cropped bottom-half for lane detection.

$$c = \frac{1}{n} \sum_{i=1}^n x_i \quad (1)$$

Lane center calculation [Ma21]

Tab. 1: Dataset summary after filtering

| Category              | Count |
|-----------------------|-------|
| With green line       | 1949  |
| Without green line    | 2259  |
| Centered green line   | 630   |
| Off-center green line | 1319  |
| Corrupted images      | 320   |

## 4.2 SVM Classification and Evaluation

Each frame was labeled based on the horizontal position of the detected lane center as **Left (-1)**, **Center (0)**, or **Right (+1)**. These labeled samples were used to train a linear Support Vector Machine (SVM), which separates the classes by finding a hyperplane that maximizes the margin between them.

A linear kernel computes similarity using the dot product, while the margin is defined as the distance between support vectors and the decision boundary:

$$K(x, x') = x \cdot x' = \sum_{i=1}^n x_i x'_i \quad (2) \quad \text{Linear kernel function [Op23a]}$$

$$\text{distance} = \frac{|\beta_0 + \beta^T x|}{\|\beta\|} = \frac{1}{\|\beta\|} \quad (3) \quad \text{Margin from support vectors [Op23a]}$$

Maximizing this margin improves the model's generalization to unseen inputs. Figure 39 shows the confusion matrix from testing, while Figure 38 illustrates how the model was implemented in CARLA for steering control.

In CARLA, the `predict_steering` function was used to apply the trained model. The function crops the lower half of the camera image, converts it to HSV, applies binary masks to extract lane pixels, and flattens the result into a 1D feature vector. This vector is fed into the SVM, which predicts the lane class and maps it to a steering angle. The vehicle then adjusts its direction accordingly, enabling real-time lane-following.

```
def predict_steering(img):
    if not hasattr(predict_steering, "_model"):
        model_path = MODEL_PATH
        if not os.path.isfile(model_path):
            predict_steering._model = None
        else:
            predict_steering._model = joblib.load(model_path)
    img_np = np.frombuffer(img.raw_data, dtype=np.uint8)
    img_np = img_np.reshape((img.height, img.width, 4))
    img_bgr = img_np[:, :, :3]
    img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)
    mask_green = cv2.inRange(img_hsv, lower_green, upper_green)
    mask_blue = cv2.inRange(img_hsv, lower_blue, upper_blue)
    combined_mask = cv2.bitwise_or(mask_green, mask_blue)
    height, width = combined_mask.shape
    bottom_half = combined_mask[int(height * 0.5):, :]
    resized = cv2.resize(bottom_half, IMAGE_SIZE)
    features = resized.flatten().reshape(1, -1)
    model = predict_steering._model
    pred_clipped = 0.0
    if model is not None:
        try:
            pred = float(model.predict(features)[0])
        except Exception as e:
            pred = 0.0
            print(f"[ERR] SVM predict failed: {e}")
        pred_clipped = pred_to_steer_value.get(pred, 0.0)
        print(f"SVM steering prediction: {pred_clipped:.3f}")
    return pred_clipped
```

Fig. 38: Predicting steering from lane input

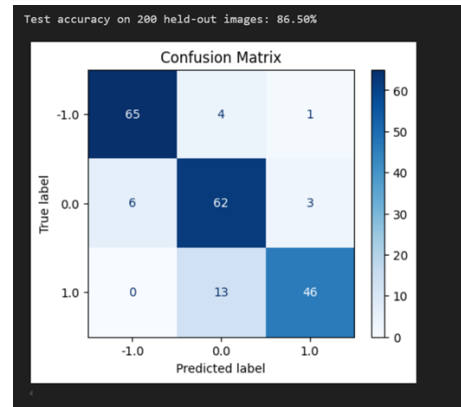


Fig. 39: Confusion matrix from testing

The SVM's core operations such as the dot product and margin maximization were implemented using scikit-learn's linear kernel. This provided an efficient and simple way to map lane pixel features into a space where steering decisions could be made accurately.

## 5 Conclusion

This project combined SVM-based lane classification and UPPAAL modeling to support autonomous lane following and truck merging. Images from the CARLA simulator were used to detect lanes and classify them as left, center, or right. A linear SVM was trained with 86.5% accuracy and used to steer the vehicle in real time.

The merging scenario was modeled in UPPAAL using timed automata and broadcast signals like lane, resume and stop. The model was verified for safety, timing, and deadlock freedom. SysML diagrams supported the design by showing structure and behavior. Also the two main challenges were input shape mismatches and unflattened arrays during prediction. These were resolved by properly formatting the input data. Overall, the system showed that combining lightweight vision-based classification with formal modeling can support safe and coordinated autonomous behavior. It is important to note that the goal of our UPPAAL model was to identify all different safety scenarios, by allowing the model to randomly choose a value for the distance to the obstacle and the safety gap between the trucks. This addition not only provided for realism into our model by testing different cases, but is also the reason why verifying the deadlock takes 160 seconds, as around 170 thousand different scenarios get tested ensuring road safety and correct configuration.

## 6 Declaration of Originality

We are Mohamed Abdo, Arya Karimi Jafari, and Mustafa Qaiser, here with declare that we have composed the present paper and work by ourselves and without the use of any other than the cited sources and aids. Sentences or parts of sentences quoted literally are marked as such; other references with regard to the statement and scope are indicated by full details of the publications concerned. The paper and work in the same or similar form have not been submitted to any examination body and have not been published. This paper was not yet, even in part, used in another examination or as a course performance. We agree that our work may be checked by a plagiarism checker.

---

06/07/2025 Lippstadt

Mohamed Abdo, Arya Jafari, Mustafa Qaiser

## Bibliography

- [As23] Asadi, Mohammad et al.: Safe and efficient cooperative on-ramp merging using connected and automated vehicles. *IEEE Transactions on Intelligent Vehicles*, 8(1):122–135, 2023.
- [Fr95] Franke, U.; Bottiger, F.; Zomotor, Z.; Seeberger, D.: Truck platooning in mixed traffic. In: *Proceedings of the Intelligent Vehicles '95. Symposium*. pp. 1–6, 1995.

- [IC23] Irshayyid, Ahmad; Chen, Jian: Comparative Study of Cooperative Platoon Merging Control Based on Reinforcement Learning. *Sensors*, 23(2):990, 2023.
- [Ma21] Mallick, Satya: Find Center of a Blob (Centroid) Using OpenCV (C++/Python). <https://learnopencv.com/find-center-of-blob-centroid-using-opencv-cpp-python/>, 2021. LearnOpenCV.
- [Op23a] OpenCV: Introduction to Support Vector Machines. *OpenCV Docs*, 2023. [https://docs.opencv.org/4.x/d1/d8a/tutorial\\_introduction\\_to\\_svm.html](https://docs.opencv.org/4.x/d1/d8a/tutorial_introduction_to_svm.html).
- [Op23b] OpenCV: OpenCV: Contour Features. *OpenCV Docs*, 2023. [https://docs.opencv.org/4.x/d1/d32/tutorial\\_py\\_contour\\_features.html](https://docs.opencv.org/4.x/d1/d32/tutorial_py_contour_features.html).